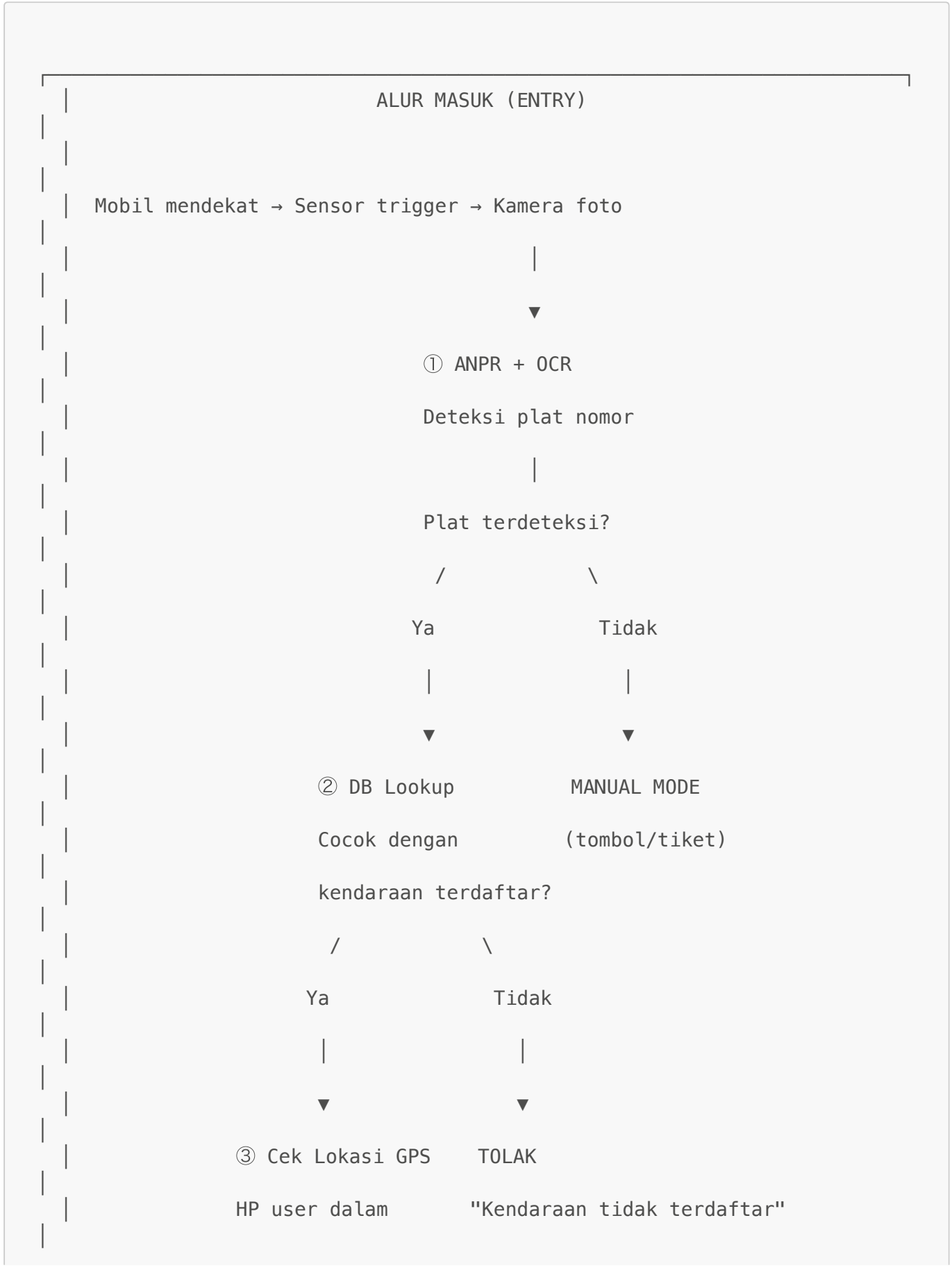
Roadmap Integrasi Sistem SmartPark (v2)

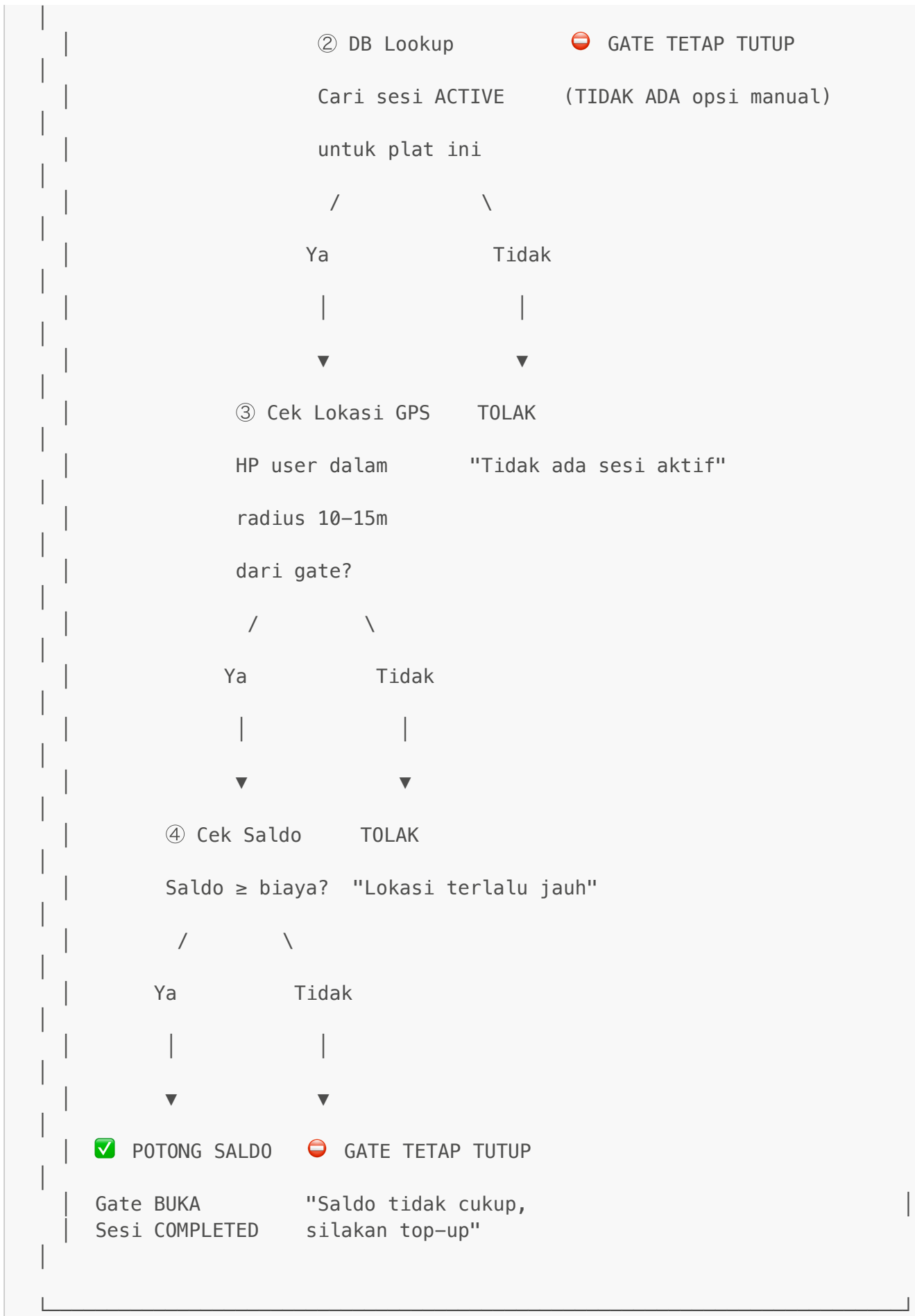
Gambaran Alur Sistem Lengkap



```
  │              radius 10–15m
  │
  │              dari gate?
  │
  │               /         \
  │
  │             Ya           Tidak
  │
  │              │            │
  │
  │              ▼            ▼
  │
  │        ✅ GATE BUKA    TOLAK
  │
  │        Sesi parkir     "Lokasi terlalu jauh"
  │
  │        dimulai
  │
  │        (status=ACTIVE)
  │
└──────────────────────────────────────────────────────────────────────────┘

                  ... Waktu parkir berjalan ...
                  ... App hitung biaya realtime ...
                  ... Jika biaya > saldo → NOTIF TOP-UP ...

┌──────────────────────────────────────────────────────────────────────────┐
  │                          ALUR KELUAR (EXIT)
  │
  │
  │
  │  Mobil ke gate keluar → Sensor → Kamera foto
  │
  │                                      │
  │
  │                                      ▼
  │
  │                             ① ANPR + OCR
  │
  │                             Deteksi plat nomor
  │
  │                                      │
  │
  │                             Plat terdeteksi?
  │
  │                              /           \
  │
  │                            Ya             Tidak
  │
  │                             │               │
  │
  │                             ▼               ▼
```

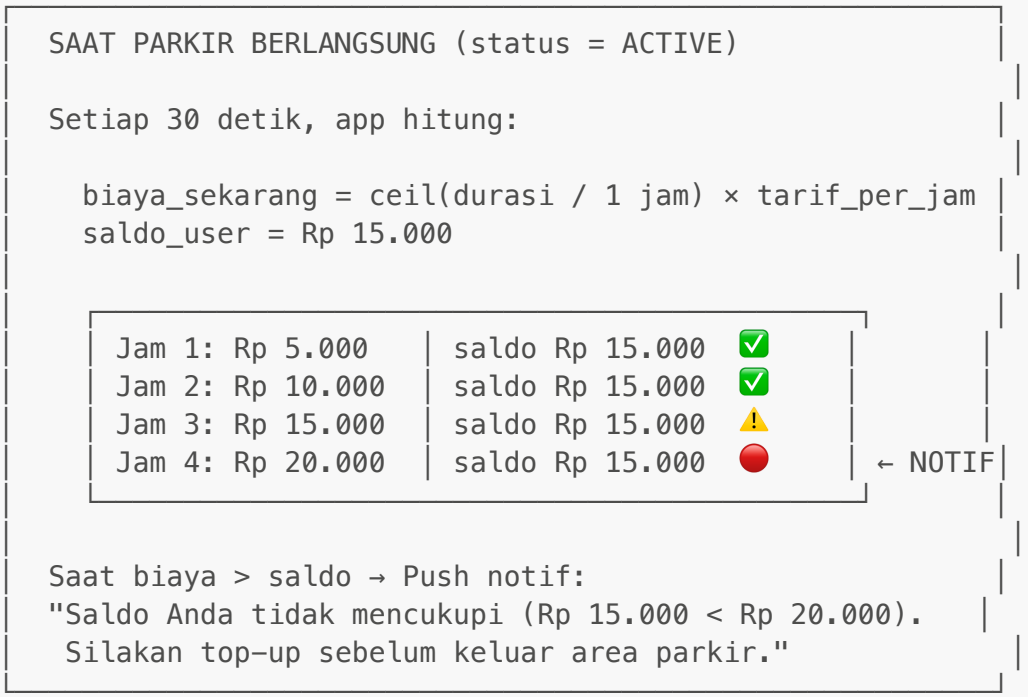

② DB Lookup
⛔ GATE TETAP TUTUP
Cari sesi ACTIVE
(TIDAK ADA opsi manual)
untuk plat ini
Ya
Tidak
③ Cek Lokasi GPS
TOLAK
HP user dalam
"Tidak ada sesi aktif"
radius 10–15m
dari gate?
Ya
Tidak
④ Cek Saldo
TOLAK
Saldo ≥ biaya?
"Lokasi terlalu jauh"
Ya
Tidak
✅ POTONG SALDO
⛔ GATE TETAP TUTUP
Gate BUKA
"Saldo tidak cukup,
Sesi COMPLETED
silakan top-up"

Perbedaan Gate Masuk vs Keluar

Aspek	Gate Masuk	Gate Keluar
ANPR + OCR	✅ Ya	✅ Ya
DB Lookup	Cek kendaraan terdaftar	Cari sesi parkir ACTIVE
Verifikasi GPS	✅ Ya (radius 10-15m)	✅ Ya (radius 10-15m)
Potong saldo	❌ Tidak	✅ Ya
Opsi manual	✅ Ya (tombol/tiket)	❌ Tidak ada
Jika gagal	Fallback manual	Gate tetap tutup

> [!IMPORTANT] **Gate keluar TIDAK memiliki opsi manual.** Semua verifikasi harus berhasil (ANPR + DB + GPS + saldo) agar gate terbuka. Ini untuk memastikan pembayaran selalu tercatat.

Notifikasi Saldo Kurang (Realtime)



> [!NOTE] Notifikasi ini terjadi **SELAMA masih parkir**, bukan di gate keluar. Tujuannya memberi waktu user untuk top-up sebelum ke gate keluar.

Yang Sudah Selesai ✅

Komponen	Status
Model ANPR (YOLOv8)	✅ Trained (`best.pt`)

Komponen	Status
OCR Engine (EasyOCR)	✅ Berjalan
API Endpoint deteksi	✅ `/anpr/detect`, `/ocr/read`
API Endpoint pipeline	✅ `/pipeline/verify`
UI Aplikasi	✅ Sudah dibuat (oleh user)

Yang Perlu Dibangun 🔨

Phase 1: Database & Data Model

Tujuan: Menyimpan data pengguna, kendaraan, dan sesi parkir.

Tabel yang Dibutuhkan:

```
┌──────────────────────────────────────────────┐
│  USERS                                       │
│  ─────                                       │
│  id            UUID (PK)                     │
│  name          VARCHAR                       │
│  email         VARCHAR (unique)              │
│  phone         VARCHAR                       │
│  password_hash VARCHAR                       │
│  balance       DECIMAL (saldo e-wallet)      │
│  created_at    TIMESTAMP                     │
└──────────────────────────────────────────────┘

┌──────────────────────────────────────────────┐
│  VEHICLES                                    │
│  ────────                                    │
│  id            UUID (PK)                     │
│  user_id       UUID (FK → users)             │
│  plate_number  VARCHAR (unique) — "B 1234 ABC"│
│  color         VARCHAR — "black" (opsional)  │
│  brand         VARCHAR — "Toyota" (opsional) │
│  is_active     BOOLEAN                       │
│  created_at    TIMESTAMP                     │
└──────────────────────────────────────────────┘

┌──────────────────────────────────────────────┐
│  PARKING_SESSIONS                            │
│  ────────────────                            │
│  id            UUID (PK)                     │
│  vehicle_id    UUID (FK → vehicles)          │
│  user_id       UUID (FK → users)             │
│  plate_number  VARCHAR                       │
│  gate_in_id    VARCHAR — "GATE-A-IN"         │
│  gate_out_id   VARCHAR (nullable)            │
```

```
│  entry_time    TIMESTAMP                                    │
│  exit_time     TIMESTAMP (nullable)                         │
│  duration_min  INTEGER (nullable, computed)                 │
│  total_cost    DECIMAL (nullable, computed)                 │
│  status        ENUM: ACTIVE / COMPLETED                     │
│  entry_photo   VARCHAR (path foto masuk)                    │
│  exit_photo    VARCHAR (path foto keluar, nullable)         │
│  entry_method  ENUM: AUTO / MANUAL                          │
│  created_at    TIMESTAMP                                    │
└─────────────────────────────────────────────────────────────┘

┌─────────────────────────────────────────────────────────────┐
│  TRANSACTIONS                                               │
│  ────────────                                               │
│  id            UUID (PK)                                    │
│  user_id       UUID (FK → users)                            │
│  session_id    UUID (FK → parking_sessions, nullable)       │
│  type          ENUM: TOPUP / PARKING_FEE / REFUND           │
│  amount        DECIMAL                                      │
│  balance_after DECIMAL                                      │
│  description   VARCHAR                                      │
│  created_at    TIMESTAMP                                    │
└─────────────────────────────────────────────────────────────┘

┌─────────────────────────────────────────────────────────────┐
│  PARKING_RATES                                              │
│  ─────────────                                              │
│  id            UUID (PK)                                    │
│  vehicle_type  VARCHAR — "car" / "motorcycle"               │
│  rate_per_hour DECIMAL — 5000                               │
│  max_daily     DECIMAL — 50000                              │
│  is_active     BOOLEAN                                      │
└─────────────────────────────────────────────────────────────┘

┌─────────────────────────────────────────────────────────────┐
│  GATE_LOCATIONS                                             │
│  ──────────────                                             │
│  id            VARCHAR (PK) — "GATE-A-IN"                   │
│  name          VARCHAR — "Gate Masuk A"                     │
│  type          ENUM: ENTRY / EXIT                           │
│  latitude      DECIMAL — -6.200123                          │
│  longitude     DECIMAL — 106.800456                         │
│  radius_meters INTEGER — 15 (default 10-15m)                │
│  is_active     BOOLEAN                                      │
└─────────────────────────────────────────────────────────────┘
```

Phase 2: API Backend — Core Business Logic

Auth & User

```
POST   /api/auth/register        → Daftar akun baru
POST   /api/auth/login           → Login, return JWT token
GET    /api/users/me             → Profil + saldo pengguna
```

Vehicle Management

```
POST   /api/vehicles             → Daftarkan kendaraan (plat)
GET    /api/vehicles             → List kendaraan user
DELETE /api/vehicles/{id}        → Hapus kendaraan
```

Gate Entry (dipanggil oleh sistem gate)

```
POST   /api/gate/entry
  Body: { image: file, gate_id: "GATE-A-IN", user_lat: -6.2, user_lon:
106.8 }

  Flow:
    ① ANPR + OCR → dapat plate_number
    ② DB Lookup → cari plate di tabel vehicles
       - Tidak ditemukan → { action: "MANUAL_REQUIRED", reason: "Kendaraan
tidak terdaftar" }
    ③ Verifikasi GPS → jarak user ke gate < radius (10-15m)?
       - Terlalu jauh → { action: "REJECTED", reason: "Lokasi terlalu jauh"
}
    ④ Semua OK → buat parking_session (ACTIVE), buka gate
       → { action: "OPEN_GATE", session_id: "...", plate: "B 1234 ABC" }
```

Gate Exit (dipanggil oleh sistem gate) — TANPA OPSI MANUAL

```
POST   /api/gate/exit
  Body: { image: file, gate_id: "GATE-A-OUT", user_lat: -6.2, user_lon:
106.8 }

  Flow:
    ① ANPR + OCR → dapat plate_number
       - Gagal deteksi → { action: "REJECTED", reason: "Plat tidak
terdeteksi" }
    ② DB Lookup → cari parking_session ACTIVE untuk plate ini
       - Tidak ditemukan → { action: "REJECTED", reason: "Tidak ada sesi
aktif" }
    ③ Verifikasi GPS → jarak user ke gate < radius (10-15m)?
       - Terlalu jauh → { action: "REJECTED", reason: "Lokasi terlalu jauh"
}
    ④ Hitung biaya → durasi × tarif_per_jam
    ⑤ Cek saldo user ≥ biaya?
```

```
        - Tidak cukup → { action: "INSUFFICIENT_BALANCE", cost: 20000,
balance: 15000 }
    ⑥ Semua OK → potong saldo, tutup sesi, buka gate
       → { action: "OPEN_GATE", cost: 15000, new_balance: 35000 }
```

Parking Session

```
GET    /api/parking/active       → Sesi parkir aktif + biaya realtime
GET    /api/parking/history      → Riwayat parkir
```

Wallet / Saldo

```
GET    /api/wallet/balance       → Cek saldo
POST   /api/wallet/topup         → Top-up saldo
GET    /api/wallet/transactions  → Riwayat transaksi
```

Phase 3: Realtime Cost + Notifikasi Saldo

Tujuan: Hitung biaya parkir secara realtime dan notifikasi jika saldo kurang.

Endpoint realtime cost:

```
GET /api/parking/active
Response:
{
  "session_id": "...",
  "plate": "B 1234 ABC",
  "entry_time": "2026-05-08T14:00:00",
  "duration_minutes": 125,
  "current_cost": 15000,
  "rate_per_hour": 5000,
  "user_balance": 15000,
  "balance_sufficient": true    ← false jika cost > balance
}
```

Logic notifikasi di app (client-side):

```
Setiap 30 detik → panggil GET /api/parking/active

Jika balance_sufficient == false:
   → Tampilkan notifikasi:
     "⚠️ Saldo tidak mencukupi!
      Biaya parkir: Rp 20.000
```

```
        Saldo Anda: Rp 15.000
        Silakan top-up Rp 5.000 sebelum keluar."
```

Phase 4: Verifikasi Lokasi GPS

Tujuan: Verifikasi user di dekat gate sebelum buka gate (masuk) atau potong saldo (keluar).

Konfigurasi Gate:

```
GATE_LOCATIONS tabel menyimpan:
  - Koordinat setiap gate (latitude, longitude)
  - Radius toleransi (default: 15 meter)
```

Formula jarak (Haversine):

```
from math import radians, sin, cos, sqrt, atan2

def haversine(lat1, lon1, lat2, lon2):
    """Hitung jarak antara 2 koordinat dalam meter."""
    R = 6371000  # Radius bumi (meter)
    dlat = radians(lat2 - lat1)
    dlon = radians(lon2 - lon1)
    a = sin(dlat/2)**2 + cos(radians(lat1)) * cos(radians(lat2)) *
sin(dlon/2)**2
    return R * 2 * atan2(sqrt(a), sqrt(1-a))

def verify_location(user_lat, user_lon, gate_id):
    gate = db.get_gate(gate_id)
    distance = haversine(user_lat, user_lon, gate.latitude, gate.longitude)
    return {
        "nearby": distance <= gate.radius_meters,
        "distance_meters": round(distance, 1),
        "max_radius": gate.radius_meters,
    }
```

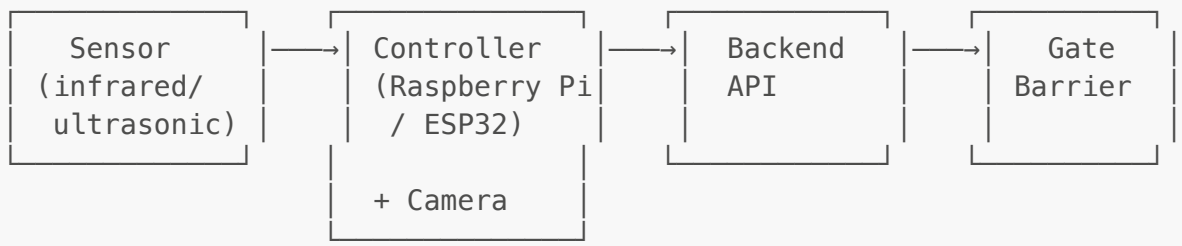
Urutan verifikasi (SELALU sama di masuk dan keluar):

```
① ANPR + OCR  →  ② DB Lookup  →  ③ GPS Radius  →  ④ Action
```

Perbedaan hanya di ④ **Action**:

- **Masuk:** Buat sesi parkir → buka gate
- **Keluar:** Potong saldo → tutup sesi → buka gate

Phase 5: Hardware Integration



Flow di Controller (Entry Gate):

```
while True:
    if sensor.detect_vehicle():
        image = camera.capture()
        user_location = get_nearest_user_location()  # Dari app via API

        response = requests.post(
            "http://backend:8000/api/gate/entry",
            files={"image": image},
            data={"gate_id": "GATE-A-IN",
                  "user_lat": user_location["lat"],
                  "user_lon": user_location["lon"]}
        )

        if response["action"] == "OPEN_GATE":
            gate.open()
            time.sleep(10)
            gate.close()
        elif response["action"] == "MANUAL_REQUIRED":
            display.show("Mode manual: ambil tiket")
            ticket.dispense()
        else:
            display.show(response["reason"])
```

Flow di Controller (Exit Gate) — TANPA MANUAL:

```
while True:
    if sensor.detect_vehicle():
        image = camera.capture()
        user_location = get_nearest_user_location()

        response = requests.post(
            "http://backend:8000/api/gate/exit",
            files={"image": image},
            data={"gate_id": "GATE-A-OUT",
                  "user_lat": user_location["lat"],
                  "user_lon": user_location["lon"]}
```

```
        )

        if response["action"] == "OPEN_GATE":
            display.show(f"Biaya: Rp {response['cost']:,}")
            gate.open()
            time.sleep(10)
            gate.close()
        elif response["action"] == "INSUFFICIENT_BALANCE":
            display.show("Saldo tidak cukup. Silakan top-up di aplikasi.")
            # Gate TETAP TUTUP — tidak ada opsi manual
        else:
            display.show(response["reason"])
            # Gate TETAP TUTUP
```

Urutan Pengerjaan (Checklist)

✅ Sudah Selesai

- [x] Model ANPR (YOLOv8 trained)
- [x] OCR Engine (EasyOCR)
- [x] API deteksi plat
- [x] UI Aplikasi

🔨 Tahap 1: Database & Auth (Prioritas Tinggi)

- [ ] Setup database (PostgreSQL / Supabase)
- [ ] Buat tabel: users, vehicles, parking_sessions, transactions, rates, gate_locations
- [ ] API auth: register, login (JWT)
- [ ] API vehicles: daftar, list, hapus kendaraan
- [ ] API wallet: cek saldo, top-up, riwayat

🔨 Tahap 2: Gate Logic (Prioritas Tinggi)

- [ ] `POST /api/gate/entry` — ANPR + OCR + DB + GPS → buka gate / manual
- [ ] `POST /api/gate/exit` — ANPR + OCR + DB + GPS + saldo → buka gate (NO manual)
- [ ] Hitung biaya parkir otomatis (per jam)
- [ ] Verifikasi GPS radius 10-15m

🔨 Tahap 3: Realtime & Notifikasi (Prioritas Sedang)

- [ ] `GET /api/parking/active` — sesi aktif + biaya realtime + cek saldo cukup
- [ ] Polling dari app setiap 30 detik
- [ ] Notifikasi saldo kurang **saat masih parkir** (bukan di gate keluar)

🔨 Tahap 4: Hardware (Prioritas Rendah — Bisa Disimulasikan)

- [ ] Setup Raspberry Pi + kamera
- [ ] Script controller gate (entry + exit)
- [ ] Integrasi sensor + barrier gate

Catatan Penting

> [!WARNING] **Gate keluar tidak memiliki opsi manual.** Semua step (ANPR → DB → GPS → Saldo) harus berhasil. Jika salah satu gagal, gate tetap tutup. Ini by design untuk menjamin pembayaran.

> [!TIP] **Untuk demo/presentasi:** Hardware bisa disimulasikan dengan Postman. Kirim foto + koordinat GPS secara manual sebagai pengganti sensor + kamera + GPS.